

Deep-Dive Explanation — D435f_UR_pose_ROSnode.py

Purpose

This node detects a single UR3 pose in 2D from a color stream using an Ultralytics YOLO pose model, draws the joints/skeleton on the image, and publishes both the annotated image and a 6×3 keypoint array (x, y, conf) for the best detection. There is no depth/3D/TF in this script—pure 2D pose.

Key Components & Defaults

- Model path & topics: `--model` (env `UR_MODEL`); input `--image-topic` (env `UR_IMAGE_TOPIC`, default `/cam2/color/image_raw`); outputs `--out-image-topic` (default `/ur3_pose/annotated`) and `--out-kpt-topic` (default `/ur3_pose/keypoints`). The CLI parser tolerates `--ros-args` so you can launch like any ROS node.
- QoS: Subscriber uses `BEST_EFFORT / KEEP_LAST / depth=5` (camera-friendly). Annotated image uses the same QoS; keypoints publisher uses queue depth 10.
- GPU/CPU: Loads the YOLO model and moves it to CUDA if available, else CPU; inference runs under `torch.inference_mode()`.

Joints, Colors, and Skeleton

- `JOINT_NAMES` = [base, shoulder, elbow1, elbow2, elbow3, wrist]
- Skeleton edges: (0,1),(1,2),(2,3),(3,4),(4,5) — drawn only if both endpoints have confidence > 0.5.
- `draw_pose()` renders lines, filled circles per joint, and labels each with its name.

Image Callback Flow

- 1) Throttle ~30 Hz to avoid backlog.
- 2) Convert ROS Image → OpenCV BGR via `CvBridge`.
- 3) YOLO predict: take `results[0].keypoints`, compute mean keypoint confidence per detection, select the best instance, extract its (6,3) array (x, y, conf), and overlay using `draw_pose()`.
- 4) Publish annotated image: reuse the input header (timestamp/frame_id).
- 5) Publish keypoints: pack (6,3) into `Float32MultiArray` with layout dims `[('joints',6,18), ('fields(x,y,conf)',3,3)]`. If nothing detected, publish an empty array.

Why Best-Instance Selection?

Choosing the detection with the highest mean keypoint confidence reduces jitter and avoids publishing multiple arrays when only one robot is expected. You can remove this restriction later to support multi-instance output.

Things to Customize First

- Weights: point `--model` to your UR3 pose weights (must be a pose model with a 6-keypoint head).
- Image topic: align `--image-topic` to your camera driver (default `/cam2/color/image_raw`).
- Confidence: raise `--conf` to filter weak detections; lower it to be more permissive.

Limitations

- 2D only: no depth or TF transforms.
- Publishes one UR3 instance per frame (by design).
- Assumes the model's class/keypoint layout matches the `JOINT_NAMES` order used for labeling.